

- evolução tecnológica
 - CPUs mais rápidas
 - discos muito maiores e mais baratos
 - mas não muito mais rápidos
 - memórias crescem exponencialmente em tamanho
 - há um parâmetro que não tem se desenvolvido tão aceleradamente
 - tempo de busca dos discos
 - SSDs são exceção (comparativamente “não têm tempo de busca”)
- desempenho dos sistemas de arquivos se tornaram gargalos
 - motivação para novos sistemas de arquivos
 - e.g., **LFS** (*Log-structured File System*)

- contexto
 - CPUs ficam mais rápidas
 - memórias RAM maiores
 - discos frequentemente incluem caches em disco
 - várias solicitações podem ser atendidas direto da cache
 - por consequência, a maior parte dos acessos será de escrita
- muitas operações de escrita são de pedaços pequenos
 - altamente ineficientes
 - e.g., uma escrita pequena leva $50 \mu s$ mas é precedida de uma busca de $10ms$ e um atraso de $4ms \rightarrow$ eficiência de $\approx 1\%$
- exemplo – escrita de um arquivo novo em UNIX
 - requer escrever i-node pro diretório, bloco do diretório, i-node pro arquivo e o arquivo em si
 - ideal é escrever imediatamente para evitar inconsistências em caso de queda

- LFS – sistema de arquivos estruturado em diário (*log*)
 - objetivo foi utilizar toda a largura de banda do disco
 - mesmo que a carga consista em pequenas operações de escrita
 - ideia é estruturar o disco inteiro como um grande diário (*log*)
 - periodicamente as operações de escrita no disco pendentes (i.e., que estão na memória) são agrupadas em um segmento e enviadas pro disco
 - tais dados são escritos logo depois do *log*
 - o segmento pode ter i-nodes, blocos de diretório, blocos de dados
 - no começo do segmento há um resumo do que ele contém
 - com um segmento médio de 1 MB já é possível usar bem a largura de banda
 - i-nodes continuam existindo (com os endereços dos blocos)
 - porém estão dispersos pelo *log* → encontrar um i-node é muito mais difícil
 - não dá pra encontrar só pelo número
 - solução: manter em disco (com cache em memória) uma tabela indexada pelo número do i-node apontando para o endereço deste no disco

- em resumo as escritas sempre ocorrem 1º na memória
 - em algum momento serão agrupadas e escritas em disco (*append* no final do diário)
- abertura de arquivo envolve encontrar a localização do i-node a partir do mapa em memória
 - após localizar o i-node é possível recuperar os endereços dos blocos
 - cada bloco estará em algum lugar do *log*
- quando parte de um arquivo é sobrescrita (ou ele é removido), o i-node é atualizado com os endereços dos novos dados
 - o que acontece com os dados antigos?
 - continuam ocupando espaço em segmentos escritos anteriormente!

- solução do LFS é usar uma thread limpadora
 - escaneia o *log* p/ compactá-lo
 1. lê o resumo de cada segmento p/ verificar quais i-nodes e arquivos estão ali
 2. confere o mapa atual de i-nodes p/ verificar se o i-node ainda é referenciado e se os blocos de arquivo ainda estão em uso
 3. caso não estejam, são descartados
 4. o que estiver em uso, vai para a memória para que seja reescrito em um novo segmento
 5. o segmento original lido é marcado como disponível
 - o disco vira um *buffer* circular
 - *thread* de escrita insere novos segmentos no início
 - *thread* limpadora remove do final
 - quando bloco é copiado p/ novo segmento, é preciso localizar o i-node, atualizá-lo, colocá-lo na memória p/ reescrita e atualizar o mapa de i-nodes p/ nova cópia
 - ainda assim o desempenho é comparável para leituras e escritas grandes
 - e supera por uma ordem de magnitude em escritas pequenas

- sistemas estruturados em *logs* são interessantes (robustez diante de falhas) mas não usados!
 - principal problema é que são incompatíveis com os sistemas de arquivos existentes
- serviram de motivação para sistemas convencionais
 - ideia é que adicionar ao sistema um diário com o que ele vai fazer (antes que o faça)
 - se ocorrer uma falha antes da efetivação das ações, o diário será usado na reinicialização para concluir o trabalho de maneira consistente
- conhecidos como **sistemas de arquivos *journaling***
 - bastante usados
 - exemplos
 - NTFS do Windows
 - ext3, ext4 e ReiserFS do Linux
 - pode ser ativado opcionalmente no macOS

exemplo – operação de remoção de arquivo no UNIX

- passos
 1. Remover o arquivo do seu diretório
 2. Liberar o i-node para o conjunto de i-nodes livres
 3. Retornar todos os blocos de disco para o conjunto de blocos de disco livres
- em operação normal a ordem os passos não importa
 - os problemas só aparecem em caso de falhas
 - e.g., se o passo 1 for concluído e logo após acontece uma falha
 - o i-node e os blocos de arquivos estarão inacessíveis
 - e também não estarão disponíveis p/ realocação (estão no limbo)
 - i.e., redução dos recursos disponíveis
 - e.g., se a falha ocorre apenas após o passo 2 → apenas os blocos serão perdidos

exemplo – operação de remoção de arquivo no UNIX

- passos
 1. Remover o arquivo do seu diretório
 2. Liberar o i-node para o conjunto de i-nodes livres
 3. Retornar todos os blocos de disco para o conjunto de blocos de disco livres
- agora vamos supor que a ordem seja diferente: passo 2 acontece primeiro e logo após ocorre uma falha
 - na reinicialização o i-node poderá ser realocado (pois estava livre)
 - porém a entrada de diretório (antiga) continua apontando pra ele → e agora ela aponta pro arquivo errado
- uma outra possibilidade é o passo 3 ocorrer 1º e ser seguido por uma falha
 - ao reiniciar, o sistema terá uma entrada de diretório apontando para um i-node que lista blocos de disco que foram marcados como livres
 - em breve, estes blocos podem ser alocados a outro arquivo
 - resultado é que dois arquivos compartilhariam os mesmos blocos!

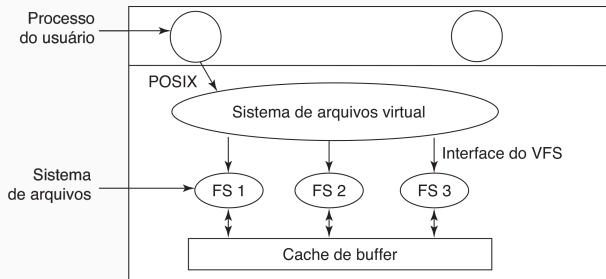
- em um sistema *journaling* há uma escrita no *log* antes de executar, de fato, as ações no disco
 - a própria entrada no *log* precisa ser escrita no disco
 - e o sistema pode tentar lê-la logo em seguida para garantir que foi de fato escrita corretamente
 - apenas depois que o registro no *log* foi confirmado é que as 3 operações em começam a serem feitas
 - se forem concluídas com sucesso, a entrada no *log* pode ser apagada
 - se ocorrerem falhas, ao reiniciar o sistema pode conferir o *log*
 - caso existam operações pendentes, elas podem ser (re)executadas até que a operação seja concluída
 - as operações podem ser reexecutadas múltiplas vezes no caso de falhas repetidas

- para que uma operação possa ser repetida quantas vezes forem necessárias, ela deve ser **idempotente**
 - i.e., não pode causar prejuízo caso seja executada mais de uma vez
- exemplos de operação idempotente
 - atualize o mapa de *bits* p/ marcar o i-node *k* ou o bloco *n* como livres
 - não há problemas se precisar ser reexecutada mais de uma vez
 - buscar um diretório e remover qualquer entrada de nome foobar
- exemplo de operação que não é idempotente
 - adicionar os blocos recentemente liberados do i-node *K* ao final da lista de i-nodes livres
 - talvez eles já estejam lá!
 - precisamos trocá-la por uma operação mais custosa: pesquisar a lista de blocos livres e incluir o bloco *n* caso ele ainda não esteja nela
- um sistema de arquivos *journaling* precisa rearranjar todas as estruturas e operações “logáveis” de forma que sejam idempotentes
 - permitirá que recuperações de falhas sejam rápidas e seguras

- outro conceito que pode ser adotado é o de **transação atômica**
 - aumenta a confiabilidade
 - um grupo de ações é transformado em uma transação
 - demarcada por `begin_transaction` e `end_transaction`
 - o sistema de arquivos deve completar todas as operações do grupo ou nenhuma delas
 - nenhuma outra combinação é permitida/possível
- NTFS faz extenso uso de *journaling*
 - lançado com o Windows NT nos anos 1990
 - raramente é corrompido por falhas no sistema
- ReiserFS foi o primeiro sistema p/ Linux a adotar *journaling*
 - pouco popular pois era incompatível com o sistema padrão da época (ext2)
 - ext3 incluiu *journaling* enquanto manteve compatibilidade com o ext2
 - ext4 foi desenvolvido nos mesmos princípios

- muito frequentemente n diferentes sistemas de arquivos são suportados por um mesmo Sistema Operacional
 - e muitas vezes também estão em uso simultaneamente no mesmo computador
- e.g., um sistema Windows cuja partição principal use NTFS mas que também possui unidades FAT-32 além de um disco ótico, etc
 - cada um destes sistemas de arquivos recebe uma letra de unidade diferente (C:, D:, etc)
 - quando um processo tenta abrir um arquivo, a partir da letra da unidade, o Windows sabe para qual sistema de arquivos deve repassar a solicitação
 - não há uma tentativa de integrar sistemas heterogêneos como um todo
- e.g., sistemas UNIX seguem a estratégia oposta
 - i.e., integrar múltiplos sistemas de arquivos em uma única estrutura
 - partição principal com ext2 montada na raiz (/), uma ext3 em /usr, uma com ReiserFS em /home e um pen-drive temporariamente em /mnt
 - usuários/processos veem uma hierarquia única (é invisível que há diferentes sistemas de arquivos)

- a maioria dos sistemas UNIX usa o conceito de **VFS (Virtual File System)**
 - ideia é abstrair a parte do sistema de arquivos que é comum a todos os sistemas de arquivos
 - essa parte do código é separada das demais
- a camada VFS chama os sistemas de arquivos abaixo dela para que façam de fato o gerenciamento



- todas as chamadas envolvendo arquivos são direcionadas ao VFS
 - são chamada POSIX padrão que formam uma camada superior
 - e.g., open, read, write, lseek, etc
- já a interface inferior lida com os sistemas de arquivos reais
 - conhecido como interface do VFS
 - é dependente de cada sistema de arquivos
 - ao criar um novo sistema de arquivos compatível com VFS, os projetistas devem implementar tais chamadas
- os sistemas abaixo do VFS não precisam estar em partições do disco local
 - motivação inicial do VFS para a Sun foi dar suporte a arquivos remotos vis **NFS** (*Network File System*)
 - enquanto o sistema de arquivos real estiver fornecendo as funções pro VFS não há problemas!
 - VFS não sabe onde estão os dados ou como é internamente o sist. de arquivos real

- estrutura da implementação
 - superbloco descreve um sistema de arquivos
 - v-node descreve um arquivo
 - diretório descreve um diretório de um sist. de arquivos
- cada um tem operações associadas (métodos) que os sistemas de arquivos reais têm que suportar
- há ainda estruturas internas do próprio VFS
 - tabela de montagem
 - descritores de arquivos, etc

Exemplo do funcionamento

- na inicialização o sistema de arquivos raiz é registrado com o VFS
 - i.e., deve fornecer os endereços das funções exigidas pelo VFS
- uma vez montado, sistema de arquivos está pronto p/ uso
- `open("/usr/include/unistd.h", O_RDONLY)`
 - VFS vê que o sistema de arquivos está montado em `/usr`
 - localiza o superbloco na lista de superblocos de sistemas montados
 - encontra o diretório-raiz do sistema de arquivos montado e examina o caminho `include/unistd.h`
 - VFS cria um v-node e faz uma chamada ao sistema real que retornará com as informações do i-node do mesmo
 - tal informação é copiada p/ o v-node (em RAM)
 - cria o ponteiro p/ as funções que operam sobre v-nodes (`read`, `write`, etc)
 - uma nova entrada apontando para o novo v-node é registrada na tabela de descritores de arquivo do processo
 - VFS retorna o descritor de arquivo p/ o processo que chamou `open`
 - finalmente o processo pode usar o descritor p/ ler, escrever ou fechar o

Exemplo do funcionamento

- operação read usando o descritor de arquivos
 - VFS localiza o v-node nas tabela de descritores e segue os ponteiros para as funções
 - i.e., ponteiros pras funções do sistema de arquivos real onde reside o arquivo
 - a função (real) responsável pelo read é chamada
 - o código do sistema de arquivos real busca o bloco solicitado
 - para o VFS tanto faz se os dados vem de um disco local ou de um sistema de arquivos remoto via rede, ou pen-drive, etc

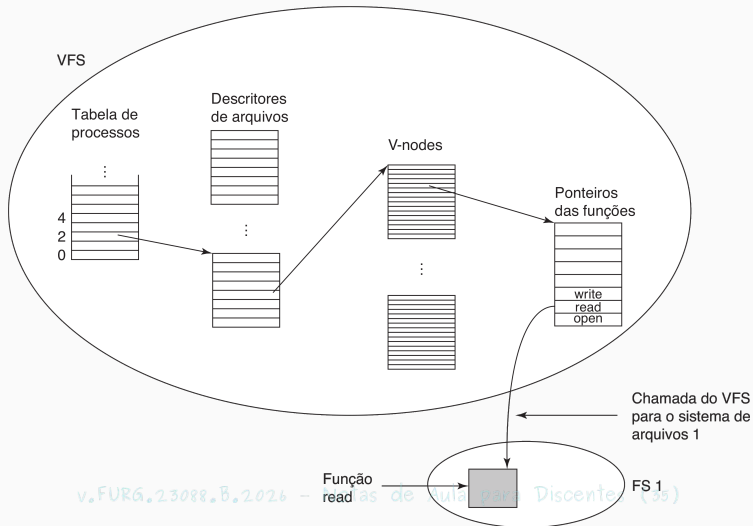
Exemplo do funcionamento

- operação read usando o descritor de arquivos
 - VFS localiza o v-node nas tabela de descritores e segue os ponteiros para as funções
 - i.e., ponteiros pras funções do sistema de arquivos real onde reside o arquivo
 - a função (real) responsável pelo read é chamada
 - o código do sistema de arquivos real busca o bloco solicitado
 - para o VFS tanto faz se os dados vem de um disco local ou de um sistema de arquivos remoto via rede, ou pen-drive, etc

Inclusão de novos sistemas de arquivos

- adicionar novos sistemas de arquivos é algo relativamente direto
 - projetistas precisam parir da lista de chamadas de funções esperada pelo VFS
 - escrever o sistema de maneira a atendê-las
 - ou criar funções adaptadoras a partir das chamadas nativas já existentes

VFS



- SSDs são baseados em memória flash
 - com características diferentes dos HDs
- não há partes móveis
 - tempos de busca e atraso rotacional não existem
 - latência de acesso é duas ordens de grandeza menor
 - não há muita diferença entre leituras aleatórias e sequenciais
- também há desvantagens
 - em SSDs leituras e escritas têm custo (bastante) diferente
 - ao contrário dos HDs
 - dezenas de μ -segundos vs milissegundos ($\approx 100x$ mais lento)
- escritas são mais lentas por duas razões
 - maneira como os *bits* são programadas (aspectos físicos)
 - para escrever um pedaço de dado é preciso antes apagar uma área maior

- diferença entre *flash page* and *flash block*
 - algo como 4KB vs alguns MB (diferença variável entre 64 e 256×)
- escrever uma *flash page* requer primeiro apagar um *flash block*
 - custa algumas centenas de μ -segundos
 - mas uma vez que o *flash block* foi apagado, podemos escrever várias *flash pages* neste bloco em sequência
 - mas não dá pra escrever fora de ordem (e.g., 0, depois 2 e depois 1)
- não dá pra sobreescrever uma *flash page*
 - é necessário apagar o *flash block* inteiro
 - teria que apagar o bloco inteiro e reescrever as páginas uma por uma
 - operação custosa
 - é melhor apenas marcar a *flash page* como inválida e reescrever o conteúdo em outro *flash block*
 - se não houver um *flash block* livre será preciso apagar algum

- há outro problema → desgaste!
 - escrever e apagar repetidamente faz com que em algum momento as células flash que guardam os *bits* não possam mais ser usadas
- células *flash* suportam alguns ciclos de “program/erase (P/E)”
 - logo, é preciso espalhar o desgaste entre as células
 - há um componente de HW que lida com o “espalhamento” do desgaste
 - FTL (*Flash Translation Layer*)
- por outro lado, o sistema de arquivos trabalha com requisições de “blocos de disco”
 - lê ou escreve um nº de bloco (linear)
 - cabe à FTL fazer a tradução

- sequência de escritas de páginas faz com que com o tempo, os blocos contenham várias páginas inválidas
 - de tempos em tempos é preciso recopiar as *flash pages* válidas para outros *flash blocks* para liberar o espaço ocupado pelas inválidas
 - i.e., temos um tipo de coleta de lixo
- necessidade de coletor de lixo faz as escritas randomicas serem mais lentas
 - e ainda temos que levar em conta (as mudanças n)os mapeamentos
- FTL tem papel fundamental
 - e seus detalhes são segredos industriais de cada fabricante

- Linux oferece o *Flash-Friendly File System (F2FS)*
 - um sistema de arquivos que leva em conta características de memórias flash
 - faz uso de comandos específicos como o TRIM que avisa ao SSD quais *flash pages* estão livres
 - caso contrário, o SSD ainda vai eventualmente identificar *flash pages* inválidas quanto o sistemas de arquivos tentasse reutilizá-las
- se baseia nos conceitos de Sistemas de arquivos estruturados em Log
 - evita escritas aleatórias
 - espalha as escritas para uniformizar desgaste
- já foi usado no Android em dispositivos como Nexus 9, Pixel 3, Huawei P9
 - não está muito claro se ainda é usado
- mais um caso de uma nova tecnologia que provoca mudanças no SO visando melhor uso dos recursos